Autoprogramación del datalogger SPQ desde microSD

White paper — diseño, hipótesis, verificación y riesgos abiertos. Firmware FWDLGARM 0.0.24 sobre STM32L496RGT6, placa SPQ_ARM_R1

Spymovil — Desarrollo de firmware

4 de noviembre de 2026 — rev. 2

Índice

1. Propósito de este documento **2**

2. El problema **2**
2.1. Las restricciones que condicionan todo el diseño . . . 2

3. Arquitectura **3**
3.1. Doble banco, sin cargador propio . . . 3
3.2. El mismo binario sirve para los dos bancos . . . 3
3.3. El formato del archivo . . . 3
3.4. Por qué cualquier versión distinta, y no sólo una superior . . . 4
3.5. El CRC: por software y no con el periférico del micro . . . 4

4. Las hipótesis, y las que se refutaron **4**
4.1. Lo que se verificó contra los documentos antes de escribir código . . . 4
4.2. La hipótesis grande que se refutó: el arranque por `BFB2` . . . 5
4.2.1. Lo que se esperaba . . . 5
4.2.2. Lo que se midió . . . 5
4.2.3. Las seis hipótesis que se descartaron, cada una con su medición . . . 5
4.2.4. Un error de lectura documental que vale como lección . . . 6
4.3. La hipótesis que reemplazó a `BFB2`: copiar el banco . . . 6
4.3.1. El argumento que decidió, y es de asimetría . . . 6
4.3.2. El orden del commit: la primera palabra va última . . . 6
4.3.3. Una dependencia que hay que conocer y no romper . . . 7
4.4. La hipótesis que falló en el código: `static inline` no alcanza . . . 7
4.5. La hipótesis sobre el renombrado: resultó obligatorio, no una optimización . . . 7

5. La marca de “firmware a prueba” **8**
5.1. El problema que resuelve, y el que no . . . 8
5.2. El criterio de “funciona” no se inventó . . . 8
5.3. Dónde vive, y por qué ahí . . . 8
5.4. Cómo opera . . . 9
5.5. Por qué la marca se pone ANTES de instalar . . . 9
5.6. Por qué avisa pero no revierte . . . 9
5.7. Una limitación que no se puede cerrar desde el firmware . . . 10

6. El disparo automático al arrancar **10**
6.1. Dónde se engancha, y las dos cotas . . . 10
6.2. La secuencia . . . 10
6.3. Tres decisiones de comportamiento . . . 10
6.4. La prórroga del watchdog, y por qué no se confía en el plazo . . . 11
6.5. El chequeo de plausibilidad, y el falso positivo que dio . . . 11

6.6. El interruptor, y por qué existe . . . 11

7. Verificación **12**
7.1. Qué se validó, y cómo . . . 12
7.1.1. El CRC, en las dos puntas y sin hardware . . . 12
7.1.2. El ciclo completo, y un número que cierra en cuatro puntos . . . 12
7.1.3. Los siete casos de falla . . . 12
7.1.4. El retorno a una versión anterior . . . 12
7.1.5. El disparo automático: cuatro hechos independientes . . . 12
7.2. Lo que NO está validado . . . 13
7.3. Riesgos abiertos y mitigaciones . . . 13

8. Una trampa operativa descubierta al validar **14**

9. Trabajo futuro **14**

10. Referencias **14**

1. Propósito de este documento

Describe **cómo está construida** la autoprogramación por microSD del datalogger SPQ: qué arquitectura se eligió, qué se dio por cierto y con qué evidencia, qué hipótesis se refutaron midiendo, cómo se verificó el resultado y qué riesgos quedan abiertos.

El documento complementa al *Manual de usuario*, que describe el procedimiento. Acá está el por qué.

Una advertencia sobre el método. Varias decisiones de este diseño salieron de hipótesis que resultaron falsas, y eso está documentado deliberadamente. En un mecanismo que reescribe el firmware de un equipo instalado a cientos de kilómetros, saber *qué se creyó y por qué era falso* vale tanto como saber qué quedó funcionando: es lo que evita volver a intentar el camino descartado.

2. El problema

Un datalogger instalado en campo necesitaba un técnico con programador y acceso físico a la placa para cambiarle el firmware. El requerimiento, planteado el 12 de octubre de 2026, fue:

> *"¿Este micro es capaz de autoprogramarse? ¿Sería posible que cuando arranque lea la microSD, si hay un firmware vea si la versión es superior a la instalada y se autoprograme?"*

Alcance acordado: **que un técnico de campo actualice un equipo con una tarjeta**. No es actualización por aire.

2.1. Las restricciones que condicionan todo el diseño

El equipo está desatendido	un fallo que deje el equipo sin arrancar cuesta una visita a cientos de kilómetros
La alimentación es insegura	batería, y el modem hunde el riel en los picos de transmisión. Un corte a mitad de la escritura es un escenario realista, no teórico
El camino de arranque está validado	188 horas de operación continua en campo validaron el arranque del equipo. Cualquier cosa que se agregue ahí pierde esa validación
No hay segunda oportunidad de diagnóstico	si el firmware nuevo no arranca, no hay consola, no hay registro, no hay nada

De la segunda y la tercera sale el criterio que ordena todas las decisiones de este documento: **la asimetría del riesgo**. No alcanza con que un camino sea más probable que otro; hay que preguntarse cuánto cuesta cada falla.

3. Arquitectura

3.1. Doble banco, sin cargador propio

El STM32L496RGT6 tiene **1024 KB de memoria de programa divididos en dos bancos de 512 KB**, y el manual de referencia habilita lo que hace viable todo esto:

> *"While executing a program code from bank 1, it is possible to perform a page erase operation on bank 2"* — RM0351, §3.3.8

O sea que **se puede escribir el banco 2 mientras el equipo funciona normalmente desde el banco 1**. El firmware completo ocupa 258 KB, la mitad de un banco.

```
BANCO 1   0x08000000   el firmware que se ejecuta
BANCO 2   0x08080000   el respaldo: acá se escribe el nuevo
```

Lo que esto compra, y es la decisión arquitectónica central: no hay estado "a medio programar". Con un solo banco, actualizar significa borrar el firmware en uso y escribir el nuevo: una ventana de 10 a 20 segundos en que el equipo **no tiene firmware válido**, y un corte de alimentación ahí lo deja muerto. Con dos bancos, la escritura larga y riesgosa ocurre sobre memoria que nadie está ejecutando.

Y no hace falta un cargador de arranque propio, lo cual evita un problema recursivo entero: un segundo proyecto a mantener, una partición a reservar, y la pregunta de cómo se actualiza el programa que no se puede actualizar.

3.2. El mismo binario sirve para los dos bancos

Es la propiedad que evita tener que compilar dos versiones o escribir código relocalizable: el silicio mapea el banco activo en `0x08000000` sea cual sea. Verificado en la nota de aplicación AN2606 ("Set Bank Swap to Bank2"), y confirmado en la práctica: el mismo archivo se escribió en el banco 2 y se ejecutó desde el banco 1 sin relinkear nada.

3.3. El formato del archivo

`FIRMWARE.BIN` = **64 bytes de cabecera + la imagen cruda**, todo en little-endian.

```
off  largo  campo
  0      4  magic "SPQF"
  4      2  hdr_ver     version del FORMATO (hoy 1)
  6      2  hdr_len     64
  8     16  fw_ver      "0.0.23", terminado en NUL
 24     16  fw_hw       "SPQ_ARM_R1", terminado en NUL
 40      4  img_len     bytes de la imagen, sin esta cabecera
 44      4  img_crc32   CRC-32 (zlib) de la imagen
 48      4  hdr_crc32   CRC-32 de los bytes 0..47
 52     12  reservado
```

Cuatro decisiones, cada una con su motivo:

1. Se usa el `.bin`, no el `.elf`. El ELF es un contenedor con cabeceras, secciones, símbolos e información de depuración —288 KB contra 176 de imagen real— y cargarlo obligaría a interpretar sus *program headers* y copiar cada segmento a su dirección. El `.bin` se copia tal cual.

2. La versión viaja en la cabecera, no en el nombre del archivo. Un nombre es manipulable y no verificable; la cabecera está respaldada por su propio CRC.

3. La versión y el modelo los extrae la herramienta del código fuente, no de argumentos de línea de comandos. Pasarlos a mano crearía una segunda fuente de verdad que se puede desincronizar del binario, y **un firmware que dice ser una versión que no es** es exactamente el error que este mecanismo tiene que hacer imposible.

4. El campo `fw_hw` no es decorativo. Impide que el firmware de otro producto entre en este equipo. Es el mismo criterio con el que el IMEI de prueba del equipo son quince ceros: hacer el error imposible en vez de confiar en que nadie se equivoque.

3.4. Por qué cualquier versión distinta, y no sólo una superior

El requerimiento original decía *"si la versión es superior"*. Se cambió deliberadamente: el equipo acepta **cualquier versión distinta**.

El argumento es operativo. Si sólo aceptara versiones mayores, **volver atrás obligaría a fabricar un número de versión falso** — y volver atrás es justo lo que se necesita cuando una actualización salió mal. Eso pondría al operador en la situación de tener que mentirle al equipo en el peor momento posible.

Lo que evita reprogramar en bucle no es la comparación de versión, sino el renombrado del archivo. Se explica en §6.3, porque es más subtil de lo que parece.

3.5. El CRC: por software y no con el periférico del micro

El STM32L4 tiene un periférico de CRC por hardware y **no se usa**. El motivo: arranca con el polinomio de Ethernet sin reflejar, y para que dé el mismo resultado que `zlib.crc32` —que es lo que usa la herramienta del lado de la PC— hay que configurar la reflexión de entrada, la de salida, el valor inicial y el XOR final. Son cuatro campos que, puestos mal, **dan un número plausible y distinto**: el mecanismo funcionaría en el banco y fallaría en campo sin que nada lo delate.

La implementación por software usa una tabla por nibble: **64 bytes de memoria** contra 1 KB de una tabla por byte, con resultado idéntico. Sobre 258 KB son milisegundos; el tiempo lo domina la lectura de la tarjeta.

4. Las hipótesis, y las que se refutaron

Esta sección es el núcleo del documento.

4.1. Lo que se verificó contra los documentos antes de escribir código

Hipótesis	Verificación	Resultado
Se puede escribir un banco ejecutando del otro	RM0351 §3.3.8, textual	**Confirmada**
El mismo binario sirve para los dos bancos	AN2606, "Set Bank Swap to Bank2"	**Confirmada**
La placa tiene doble banco activo	`FLASH_OPTR = 0xFFEFF8AA` leído por SWD: bit 21 (`DUALBANK`) en 1	**Confirmada por medición**
Hay margen de memoria	imagen Release de 160 KB sobre un banco de 512 KB	**Confirmada**

La decodificación del registro de opciones se validó cruzándola contra **tres valores que ya se conocían de antes** (`IWDG_SW`, `BOR_LEV`, `RDP`), los tres coincidentes. Eso no es ceremonia: en este proyecto un volcado de registros mal decodificado ya había costado tres días de diagnóstico dirigido al lugar equivocado, y desde entonces toda decodificación se controla contra algo conocido.

4.2. La hipótesis grande que se refutó: el arranque por BFB2

El diseño original era elegante y resultó **falso para esta placa**.

4.2.1. Lo que se esperaba

El STM32L4 tiene un bit de opción, BFB2 (*Boot From Bank 2*). La idea era que instalar un firmware fuera **escribir un solo bit**:

```
escribir el banco 2  ->  verificarlo  ->  poner BFB2 = 1  ->  reset
```

La atracción era enorme: una escritura de un bit es **atómica** desde el punto de vista del equipo. Hasta ese instante arranca el viejo; después, el nuevo. El estado intermedio no existe. Y la nota de aplicación AN2606 (figura 102, la del L496 exacto) describía además un **repliegue automático del silicio**: si la primera palabra del banco 2 no apunta a memoria RAM, el microcontrolador arranca del banco 1 de todas formas.

4.2.2. Lo que se midió

Con `BFB2 = 1` **el microcontrolador no ejecuta una sola instrucción**. Y lo que cierra el diagnóstico no es la ausencia de síntomas, sino cuatro registros en su valor de reset:

```
FLASH_ACR     = 0x00000600    LATENCY = 0
RCC_CFGR      = 0x00000000    sin PLL, reloj en MSI
RCC_CR        = 0x00000063    HSI APAGADO
SYSCFG_MEMRMP = 0x00000000    FB_MODE = 0: el intercambio de bancos NI SE APLICO
```

El argumento decisivo es la latencia de flash. El cargador interno del microcontrolador corre a 72 MHz, y a esa frecuencia el silicio exige latencia mayor o igual a 3. Si el cargador estuviera ejecutando, `LATENCY` **no podría** valer 0. Y `FB_MODE = 0` dice que el intercambio de bancos —lo primero que BFB2 debería hacer— tampoco ocurrió.

Es decir: la lectura correcta **no** es *“el cargador se queda esperando comandos”*, que fue el marco de trabajo durante todo el diagnóstico, sino que **el micro no arranca en absoluto**.

4.2.3. Las seis hipótesis que se descartaron, cada una con su medición

Sospechoso	Cómo se descartó
El binario del banco 2	se escribió una **copia bit a bit del banco 1** y tampoco arranca
La herramienta que genera el archivo, o el grabado	el banco 2 releído por SWD coincide con el archivo: **0 bytes distintos en 256.080**
El puntero de pila fuera de la RAM por un byte	se corrigió de `0x20040000` a `0x2003FFF8` y **sigue sin arrancar**. El cambio se conservó igual: el criterio de ST es correcto y cuesta 8 bytes
El reloj que deja el cargador	**no alcanza a configurarlo**: `RCC_CR` lo muestra apagado
El pin `PH3-BOOT0` flotando	se forzó el arranque desde memoria principal por bits de opción (`nSWBOOT0 = 0`) y **el síntoma es idéntico**. Además Pablo confirmó que `PH3` **tiene un pull-down de 10 K**, así que el nivel era firme
El mecanismo de “memoria vacía” del silicio	el flag `FLASH_ACR.EMPTY` leído en los dos casos: **vale 0**

La causa última no se encontró, y el documento no pretende lo contrario. **Lo que se sabe con certeza es que no es algo que el firmware pueda arreglar**, y eso alcanzó para abandonar el camino.

4.2.4. Un error de lectura documental que vale como lección

Durante el diagnóstico se advirtió que, con `BFB2 = 1`, el cargador interno podía tomar los pines `PA5`, `PA6`, `PA7` y `PB12..PB15` — que en esta placa son, entre otros, la **alimentación de la electroválvula** y el bus I2C. La advertencia se basó en la tabla 172 del RM0351.

Era falsa. Esa tabla describe la configuración de periféricos de la rama *"Continue Bootloader execution"* del diagrama de flujo, que **sólo se recorre si ningún banco tiene código válido**, o sea cuando el equipo ya está inutilizable. En operación normal el cargador verifica y salta sin tocar un pin.

> **La lección: una tabla leída fuera de su diagrama de flujo puede decir lo contrario de lo que dice.** Es de la misma familia que el volcado de registros mal decodificado — la diferencia es que acá el documento correcto estaba a dos páginas.

4.3. La hipótesis que reemplazó a `BFB2`: copiar el banco

Decidido el 30 de octubre de 2026. El firmware nuevo se pone en marcha **copiándolo al banco 1**, no saltando a él.

Se evaluaron dos caminos:

Camino A — salto por software. El banco 1 arranca siempre, lee una marca, y si corresponde escribe `SYSCFG_MEMRMP.FB_MODE` y salta al banco 2 desde una función en RAM.

Camino B — copiar el banco 2 al banco 1 y reiniciar.

4.3.1. El argumento que decidió, y es de asimetría

Si falla…	Cuesta
la copia (camino B)	82 microsegundos de ventana crítica, y el estado intermedio es **recuperable**
el arranque (camino A)	el equipo no arranca, **en campo, a cientos de kilómetros**

El camino A mete código en el único camino que 188 horas de operación de campo habían validado. El camino B no lo toca en absoluto: el equipo arranca exactamente igual que antes, y todo lo nuevo corre dentro de un comando con el técnico presente.

Es la misma lógica con la que se eligió el tiempo de la electroválvula: un tiempo de más no cuesta nada, uno de menos deja una válvula a medio camino que el firmware cree cerrada.

4.3.2. El orden del commit: la primera palabra va última

Es el diseño, no un detalle de implementación:

```
1. borrar el banco 1 entero (mass erase)           22 ms
2. copiar banco 2 -> banco 1, MENOS el primer doubleword    ~2,6 s
3. VERIFICAR releyendo
4. recién ahí, el primer doubleword                82 us
5. reset
```

Mientras ese primer doubleword —el puntero de pila inicial y el vector de reset— no esté escrito, la dirección `0x08000000` lee `0xFFFFFFFF`, y entonces entra un mecanismo del silicio:

> *"If the first flash memory location is found empty, the flash empty check mechanism forces boot from the system memory"* — DS11585, §3.8

O sea que el estado intermedio **no es un equipo muerto: es un equipo en el cargador interno de ST**, recuperable por SWD o por puerto serie.

Si se corta la alimentación…	Qué queda
antes del borrado	el firmware viejo, intacto
durante los ~2,6 s de la copia	memoria vacía, y arranca el cargador de ST: **recuperable**
durante los **82 us** finales	el único caso realmente malo
después	el firmware nuevo

El borrado usa *mass erase* de banco y no borrado por página: 22 ms contra 2,8 segundos (128 páginas por 21 ms cada una). Los tiempos son del datasheet DS11585, no estimaciones: `tME` = 22,13 ms típico, `tPROG` de un doubleword = 81,69 us.

4.3.3. Una dependencia que hay que conocer y no romper

Ese mecanismo de "memoria vacía" está activo porque el bit de opción `nSWBOOT0` vale 1. El datasheet dice explícitamente que ponerlo en 0 lo desactiva.

Durante el diagnóstico de `BFB2` ese bit se cambió a 0 como experimento, y se **devolvió a 1** al refutarse la hipótesis. Eso resultó necesario por una segunda razón que en ese momento no se conocía: es la red de seguridad del camino B.

> **No cambiar `nSWBOOT0`.** Queda anotado como palanca conocida e inocua para una revisión futura de la placa, pero con el camino B actual tiene una función.

4.4. La hipótesis que falló en el código: `static inline` no alcanza

Las tres funciones que hacen la copia tienen que vivir en **memoria RAM**, porque borran el banco desde el que se ejecutan. Dos de ellas se escribieron `static inline`, confiando en que el compilador las incorporaría al cuerpo de la función que sí estaba marcada para RAM.

GCC con `-O0` no incorpora nada. El resultado:

```
20000570  prvCommitEnRam       <- en RAM, correcto
0800af3c  prvEsperarFlash      <- EN FLASH, llamada por un salto indirecto
0800af78  prvProgramarDw       <- idem
```

Con el banco 1 ya borrado, ese salto es una excepción de la que no se vuelve. **Compiló sin un solo aviso**, y el único síntoma habría sido un equipo que no vuelve a arrancar.

Quedaron dos defensas, y las dos son necesarias por razones distintas:

1. Un verificador en el proceso de compilación (`Tools/verificar_commit_en_ram.sh`), que chequea tres cosas sobre el binario ya enlazado:

- que las tres funciones vivan en direcciones `0x2000xxxx`;
- que **ninguna llamada salga de la RAM** — el compilador genera saltos que nadie escribió;
- que ningún literal se lea de la memoria de programa.

El verificador está probado contra un caso que debe fallar: quitándole la marca de RAM a una función a propósito, informa el error y devuelve código de salida 1. Un verificador que nunca se vio disparar da falsa seguridad.

2. Una comprobación en tiempo de ejecución. Antes de tocar la memoria, el commit verifica las tres direcciones y aborta sin borrar nada si alguna no está en RAM. Convierte un error fatal en un mensaje.

4.5. La hipótesis sobre el renombrado: resultó obligatorio, no una optimización

Al diseñar la etapa automática, el renombrado del archivo a `FIRMWARE.OK` se justificaba por costo: evitar releer 258 KB y recalcular su CRC en cada arranque (unos 2 segundos con el riel de la tarjeta encendido).

Al implementarlo quedó claro que es una condición de corrección. El razonamiento ya estaba escrito en el encabezado del módulo —*"lo que evita reprogramar en bucle no es la comparación: es que el archivo se renombra"*— pero recién al construirlo se entendió el **por qué**:

La versión que se compara vive en la **cabecera del archivo**, y nada garantiza que diga la verdad sobre la imagen que ese archivo contiene. El generador de casos de prueba del propio proyecto fabrica exactamente eso: un archivo cuya cabecera dice `9.9.99` sobre la imagen de la versión `0.0.22`.

```
un archivo así, llamado FIRMWARE.BIN, sin renombrado:
   aplica -> arranca -> "hay firmware nuevo" (9.9.99 contra 0.0.22) -> aplica -> ...
   13 s de escritura de memoria de programa y un reset, EN CADA ARRANQUE, para siempre
```

Eso es un equipo inutilizado por un archivo mal fabricado, y **la comparación de versión no lo puede evitar**.

Consecuencia de diseño: si el renombrado falla, no se instala. Abortar es barato y deja todo en un estado conocido —el banco 1 intacto, el banco 2 con la imagen ya verificada— y el técnico completa con un comando. La alternativa sería instalar sin red de contención contra el bucle.

5. La marca de "firmware a prueba"

5.1. El problema que resuelve, y el que no

Sí: el firmware nuevo **arranca pero no funciona**	es el caso de campo: el técnico se fue, el equipo "anda", y pasan meses sin que nadie se entere
No: el firmware nuevo **no arranca** (falla antes del planificador de tareas)	ahí no corre nada, así que ningún mecanismo en software puede ayudar. Hace falta programador o el cargador de ST. **No se promete lo contrario**

5.2. El criterio de "funciona" no se inventó

Es **una sesión con el servidor que llega a transmitir**, o sea la transición al estado `wanONLINE_DATA` de la máquina de estados del modem.

ATENCIÓN: la condición es exactamente ésa, no que el servidor conteste `CONFIG=OK`. `fw_prueba_confirmar()` se llama al final de `prvEstadoOnlineConfig()`, así que se ejecuta cuando el `PING` volvió y el frame de configuración se procesó — **también si el servidor pidió bloques que no se satisficieron**, que es el caso normal cuando un hash difiere. Hasta el 2026-11-04 el banner del equipo anunciaba `CONFIG=OK`: una condición más estricta que la que el código aplica. Se corrigió el **mensaje**, no la lógica, porque la lógica es la correcta: lo que interesa comprobar es que el equipo llegó hasta ahí, no que su configuración coincida.

Eso ejercita, de una sola vez: el modem y su enlace, el bus I2C, la memoria EEPROM, el reloj de tiempo real y el conversor analógico. **Es una prueba más completa que cualquier autodiagnóstico que se pudiera escribir**, y es trabajo que el equipo hace igual.

5.3. Dónde vive, y por qué ahí

EEPROM externa, dirección `0x00400`, 22 bytes:

```
typedef struct __attribute__((packed)) {
    uint32_t ulMagic;                /* "PRUB" */
    char     pcVersionPrevia[16];    /* a la que se volvería */
    uint8_t  ucArranques;            /* sin confirmar */
    uint8_t  ucChecksum;
} fw_prueba_t;
```

Se evaluaron tres ubicaciones:

Dónde	Veredicto
SRAM del reloj de tiempo real (respaldada por pila)	**Descartada.** Si falla la pila el equipo revierte **en silencio**, y esa memoria ya falló dos veces en este proyecto: la tabla de asignación de datos, y el contador de lotes
SRAM2 del micro, sin inicializar	**Peor**: no hace falta que falle nada, basta un corte de alimentación
EEPROM `0x00400`	**Elegida.** Ya es dependencia crítica (la configuración vive ahí), así que no agrega un punto de falla nuevo; y si fallara **se grita**: sumas de verificación malas, configuración de fábrica, y el servidor pidiendo reconfigurar los cinco bloques

No va en los primeros bytes: `0x00000` es el bloque de configuración base, y moverlo invalidaría las configuraciones ya grabadas en los equipos instalados.

La suma de verificación cubre el rango con `offsetof( fw_prueba_t, ucChecksum )` y **no** con `sizeof - 1`. No es cosmético: el mismo error, en la tabla de asignación de la memoria de datos, hacía que la tabla se declarara inválida en cada arranque y, peor, que el mensaje **acusara a la pila del reloj**, que estaba perfecta. `sizeof - 1` coincide sólo cuando el campo es literalmente el último byte, y eso es una propiedad del relleno que agrega el compilador, no del diseño.

5.4. Cómo opera

1. La pone el instalador —tanto el comando manual como el automático— **justo antes de copiar**, con la versión que se está reemplazando.
2. Cada arranque incrementa el contador y lo anuncia en el banner y en el registro de eventos.
3. La borra `fw_prueba_confirmar()`, llamada desde la máquina de estados del modem al entrar en `wanONLINE_DATA`. Es **idempotente**: sin marca no hace nada, así que no cuesta nada llamarla en cada sesión.
4. A los 5 arranques sin confirmar, el aviso se vuelve explícito.

Son arranques y no tiempo, deliberadamente: un firmware que se cuelga provoca reinicios por watchdog y gasta los cinco rápido; uno que anda confirma en el primero.

El contador **no se deja desbordar**: con un `uint8_t` dando la vuelta, un equipo que nunca confirma volvería a “recién instalado” cada 256 arranques y el aviso se apagaría solo.

5.5. Por qué la marca se pone ANTES de instalar

Es una asimetría más:

Marca puesta y la instalación falla	el costo es **un aviso de más** en el arranque siguiente
Instalación hecha y marca sin poner	el firmware nuevo queda **sin vigilancia**, que es justo lo que la marca viene a evitar

5.6. Por qué avisa pero no revierte

Después de instalar, **los dos bancos tienen el firmware nuevo**: la copia pisó el banco 1 y el banco 2 es la fuente. La imagen anterior **ya no existe dentro del equipo**. Revertir sin tenerla sería prometer algo que no se puede cumplir.

La extensión que lo resolvería está diseñada y no construida: que el instalador guarde la imagen saliente en la tarjeta como `FWPREV.BIN` antes de copiar. Son 258 KB en una tarjeta de gigabytes, y el retorno pasaría a ser el mismo procedimiento ya validado. **Depende de que haya tarjeta**, así que no sustituye al aviso: lo complementa.

5.7. Una limitación que no se puede cerrar desde el firmware

Bajar firmware con el programador **no borra la marca**, así que un equipo reprogramado por cable puede anunciarse "a prueba" de una actualización que no hizo. Es inofensivo y se borra con un comando. **No tiene solución desde el firmware**: un binario no tiene forma de saber cómo llegó a la memoria.

6. El disparo automático al arrancar

6.1. Dónde se engancha, y las dos cotas

Una línea en el arranque de la tarea de consola, y la posición está acotada por los dos lados:

Después de	el bus I2C y la EEPROM (para la marca), la inicialización de la tarjeta, y **el banner** — si el equipo se va a reprogramar, lo último que tiene que haberse leído es qué versión estaba corriendo
Antes de	el prompt de la consola, porque después de esta llamada puede no haber prompt: el equipo se reinicia

Va **después** del aviso de "a prueba" a propósito: ese aviso es sobre el firmware que está corriendo en ese momento, así que tiene que salir aunque el arranque termine reemplazándolo.

Y de eso sale una propiedad que no se buscaba: **el retorno automático sale gratis**. Si la marca se pasó del límite y el técnico dejó en la tarjeta el firmware anterior, este mismo camino lo repone sin que nadie vaya al sitio.

6.2. La secuencia

```
1. examinar            <- el CRC del archivo ENTERO, antes de tocar memoria
2. prórroga del watchdog
3. escribir el banco 2 <- y verificarlo RELEYENDO la memoria
4. ¿parece una tabla de vectores?
5. RENOMBRAR a FIRMWARE.OK      <- antes de instalar, y obligatorio (§6.3)
6. la marca de "a prueba" + el registro de eventos, volcado
7. instalar  ->  no vuelve
```

El paso 5 va antes del 7 porque el 7 no vuelve.

6.3. Tres decisiones de comportamiento

Se calla si no hay tarjeta o no hay archivo	es el caso de todos los arranques de todos los equipos. Una línea por algo que nunca pasa es ruido que tapa lo que importa
Cualquier otro problema se dice y no se instala	archivo corrupto, de otro producto, CRC mal. El equipo arranca normal, pero un técnico que dejó una tarjeta y se fue no tiene otra forma de enterarse. Va también al registro de eventos

El chequeo de “parece un firmware” avisa pero no impide	ver §7.4

6.4. La prórroga del watchdog, y por qué no se confía en el plazo

Escribir el banco son unos 13 segundos sin ningún punto donde reportar actividad. El plazo de vigilancia por tarea son 90 segundos, así que entra con margen — **y se pide la prórroga igual**.

El motivo es de acoplamiento: depender de esos 90 segundos sería depender de un número que está puesto para otra cosa. Si mañana el plazo baja, esto se rompe por un motivo que no se parece en nada a su causa. Es el mismo criterio con el que el formateo de la tarjeta y el volcado de la ventana de datos piden su propia prórroga.

6.5. El chequeo de plausibilidad, y el falso positivo que dio

Antes de instalar se mira si el banco 2 **parece una tabla de vectores**: el puntero de pila dentro de la RAM y alineado, y el vector de reset dentro de la memoria de programa y **impar** (el bit 0 es el bit de modo Thumb; un vector par provoca una excepción al primer salto).

El criterio anterior era otro y daba un falso positivo. Comparaba los 16 bits altos del puntero de pila del banco 2 contra el del banco 1, razonando que el banco 1 es el que está corriendo y por lo tanto su valor es por definición aceptable.

Medido en banco: la versión `0.0.20` tiene su pila en `0x20040000` y la `0.0.21` en `0x2003FFF8`. El comando declaró *“no parece un firmware válido”* **sobre la imagen que arrancó y quedó funcionando perfecta**. Los dos valores son correctos: la pila **decrece** desde su valor inicial, así que el tope de la RAM es un valor legítimo — y es justo el borde donde los 16 bits altos cambian.

Y además la pregunta había dejado de ser la correcta: ese chequeo existía para avisar que el cargador de `BFB2` caería al banco 1 en silencio. Con `BFB2` abandonado y el instalador copiando bytes, el cargador no decide nada.

> **Un mensaje que manda a dudar de un archivo bueno cuesta más que no tener mensaje.** Por eso este chequeo avisa y no impide: el CRC ya probó que la memoria tiene lo mismo que el archivo, y lo único que esto mira es si el archivo parece un programa.

6.6. El interruptor, y por qué existe

`FW_UPDATE_AUTO` en el encabezado del módulo. En 0, la función sale en su primera línea y el ciclo vuelve a ser el que tipea un técnico.

Las etapas manuales **no tocan el camino de arranque** —todo corre dentro de un comando, con el técnico presente— y la automática sí, y el arranque es lo único que validaron las 188 horas de campo. Con el interruptor, la versión que *trae* el código no cambia nada y encenderlo es una decisión aparte. Es el mismo criterio con el que se incorporó el watchdog: una vez que el equipo se reprograma solo, un error ahí se ve desde afuera como *“la placa no arranca”*.

El `#if` vive en el `.c` y no en el llamador. Así la tarea de consola no necesita ningún preprocesador, y no se puede repetir una trampa que en este proyecto ya mordió tres veces: un `#if` sobre una constante cuyo `#include` está más abajo en el archivo se evalúa como 0 y compila el bloque equivocado, sin decir nada.

Las dos variantes se compilan antes de cada entrega. La variante que no se compila es justo la que esconde errores. Costo: **2.480 bytes** de memoria de programa.

7. Verificación

7.1. Qué se validó, y cómo

7.1.1. El CRC, en las dos puntas y sin hardware

Con la función extraída del propio `.c` del firmware, no con una copia:

- los cinco vectores canónicos, incluido `"123456789"` -> `0xCBF43926`;
- **por tramos de 7 bytes contra de una vez**: idéntico, y es como lo usa el firmware (bloques de 512);
- sobre un `FIRMWARE.BIN` real: el código C del firmware y `zlib.crc32` de Python dan el mismo número.

7.1.2. El ciclo completo, y un número que cierra en cuatro puntos

El 30 de octubre de 2026 se corrió el ciclo entero desde la tarjeta. **El CRC `0x24C473F4` es, a la vez:**

1. el que declara la cabecera del archivo en la tarjeta;
2. el que el firmware **releyó de la memoria** después de escribir el banco 2;
3. el de los 256.080 bytes leídos **por SWD** del banco 2;
4. el de los mismos bytes leídos por SWD del banco 1 después de instalar.

Los dos bancos quedaron además con la misma suma MD5. **El programador y el firmware son instrumentos independientes y dicen lo mismo.**

7.1.3. Los siete casos de falla

Fabricados con `Tools/mkfw_casos.py`, que parchea un archivo válido:

Archivo	Veredicto obtenido
texto plano	`no es un firmware de este mecanismo (magic)`
un byte de la cabecera	`el header esta CORRUPTO`
largo declarado 8 bytes menor	`el largo no cierra, o no entra en el banco`
otro modelo de hardware	`es el firmware de OTRO producto`
un byte de la imagen	`la imagen esta CORRUPTA`
versión `9.9.99`	`hay un firmware nuevo y valido`
el mismo archivo, después de instalarlo	`es la misma version que ya esta corriendo`

El séptimo no se fabricó y es el más fuerte de todos. Es el mismo archivo, bytes idénticos, que veinte minutos antes había dicho *"hay un firmware nuevo y válido"*. Dos veredictos opuestos sobre los mismos bytes prueban que la comparación de versión es real y no decorativa — algo que ningún archivo hecho a propósito puede demostrar.

7.1.4. El retorno a una versión anterior

Salió validado sin buscarlo: el archivo de la tarjeta era de la versión `0.0.20` y el equipo corría `0.0.21`, así que el equipo fue **hacia atrás**. Es la regla de aceptar cualquier versión distinta, ejercitada.

7.1.5. El disparo automático: cuatro hechos independientes

El 31 de octubre de 2026 se bajó por SWD un firmware `0.0.22` con el automático encendido, con un `FIRMWARE.BIN` de la versión `0.0.20` en la tarjeta, y **no se tipeó nada**.

	Lo que quedó probado
El equipo quedó corriendo **`0.0.20`**	y lo que se le bajó con el programador fue el `0.0.22`
La fecha de compilación es `Oct 30 2026 15:01:46`	la del `0.0.20`, y **no se puede falsificar**: sale de `__DATE__`/`__TIME__`
El listado de la tarjeta muestra **`FIRMWARE.OK`** y **ningún `FIRMWARE.BIN`**	el renombrado ocurrió
El arranque siguiente anunció `a prueba, la version anterior era 0.0.22`	la marca la puso el propio camino automático, con la versión que corría en ese momento

Y el bucle quedó cortado, medido: después de instalar, el examen de la tarjeta pasó de **1996 ms a 714 ms**. No es sólo que no reinstala: dejó de pagar la lectura de 258 KB y su CRC en cada arranque.

ATENCIÓN: esos 714 ms son lo que tarda el COMANDO, no el mecanismo. El tiempo que informa incluye las ~13 líneas que imprime antes, que a 9600 baudios son unos 610 ms. Separado con dos medidas —un archivo inexistente (712 ms) contra uno de 264 KB que sí se lee y verifica (2035 ms)— el costo real de examinar la tarjeta sin encontrar el archivo son **unos 100 ms**, y eso coincide con los 101 ms medidos al arrancar. **El disparo automático cuesta ~100 ms por arranque.**

Una confirmación cruzada que no se buscaba: el `0.0.20` que quedó corriendo imprime el veredicto **viejo** del chequeo de plausibilidad, el que se corrigió el día anterior. Un binario no puede imprimir un mensaje que su código no tiene, así que eso prueba por sí solo que el firmware en ejecución es genuinamente el de la tarjeta.

7.2. Lo que NO está validado

Un soak de 24 h sobre el binario final	es lo único que separa esto de campo. El automático es lo primero que toca el camino de arranque desde las 188 horas de operación real. **Previsto para el lunes siguiente a esta revisión**
La confirmación **automática** de la marca	el código está puesto y llamado desde la máquina de estados del modem, pero requiere que el modem se registre en la red. En banco no hubo cobertura. Hoy la marca se borra a mano
El camino *“misma versión, renombrar”*	necesita un `FIRMWARE.BIN` en la tarjeta, y tras la validación el archivo quedó renombrado
El camino *“archivo no aplicable”* dentro del automático	ídem: el automático sólo mira `FIRMWARE.BIN`
El camino *“el renombrado falla”*	es el que decide no instalar. No hay forma barata de provocarlo: haría falta una tarjeta protegida contra escritura
El camino de **excepción Modbus** del instalador	no aplica a este mecanismo; se menciona por completitud del módulo

7.3. Riesgos abiertos y mitigaciones

Riesgo	Mitigación actual
El automático toca el camino de arranque	el interruptor `FW_UPDATE_AUTO`, y el hecho de que sin archivo en la tarjeta el mecanismo entero son ~100 ms

Riesgo	Mitigación actual
Un corte de alimentación en la ventana de 82 us	el estado resultante arranca el cargador de ST, recuperable por SWD o serie
El instalador puede cortarle la alimentación al modem a mitad de su arranque	mismo riesgo que el comando manual, ya corrido varias veces sin consecuencias. La alternativa —hacerlo antes de crear las tareas— dejaría el mecanismo sin consola, sin EEPROM y sin tarjeta
El enlazador no impide que la aplicación crezca sobre su propio respaldo	hoy el script declara 1024 KB, o sea los dos bancos. Con 258 KB no se cruza, pero conviene acotarlo a 512 KB. **Pendiente**
El archivo no está firmado	el CRC detecta corrupción, no manipulación. Quien tenga acceso físico puede reprogramar por SWD de todas formas, así que una firma no agregaría seguridad real hoy

8. Una trampa operativa descubierta al validar

No es del mecanismo, pero afecta a cualquiera que lo pruebe: **al bajar firmware con la herramienta de línea de comandos en modo `mode=UR`, el microcontrolador queda retenido en reset** y el firmware no arranca al terminar la descarga.

El síntoma es *"el equipo parece muerto"*: la descarga informa éxito y después la consola no contesta nada, ni el banner. Durante esta validación costó veinte minutos y **dos diagnósticos equivocados**, los dos leyendo "la consola no contesta" como "el firmware falló".

Lo cerró un registro:

```
RCC_CSR = 0x04000600     PINRSTF = 1, y el bit LSIRDY en 0
                    -> el watchdog nunca arrancó -> la tarea de control NUNCA CORRIO
                         (con el firmware corriendo: 0x00000602)
```

Se suelta con `-hardRst`, que pasó a ser parte del procedimiento documentado. Es de la misma familia que el pozo conocido de OpenOCD —que deja la línea de reset activa y la placa no arranca ni cortándole la alimentación—, con la diferencia de que acá lo deja la herramienta oficial.

> **La lección de método:** el equipo *parecía* roto y lo único que lo desmintió fue un registro leído por SWD. En un diagnóstico, un instrumento que mide el estado real vale más que cualquier cantidad de razonamiento sobre los síntomas.

9. Trabajo futuro

Soak de 24 h	lo único que falta para campo
Confirmación automática de la marca	requiere cobertura de red en el banco
`FWPREV.BIN`: guardar la imagen saliente	convierte el retorno en el mismo procedimiento ya validado
Acotar el enlazador a 512 KB	para que la aplicación no pueda crecer sobre su propio respaldo
Actualización por aire	el modem sólo tendría que dejar el archivo en la tarjeta: **el resto del mecanismo no cambia**. Es la propiedad que justificó separar "dejar el archivo" de "instalarlo"

10. Referencias

RM0351	STM32L4 Reference Manual. §3.3.8 (escritura entre bancos), §3.7 (bits de opción)
AN2606	STM32 microcontroller system memory boot mode. Figura 102 (patrón 6, el del L496), tabla de periféricos del cargador
DS11585	STM32L496xx datasheet. §3.8 (mecanismo de memoria vacía), tiempos de borrado y programación
Código	`Application/tasks/fw_update.{h,c}`, `Application/tasks/fs_sd.{h,c}`, `Application/tasks/tkCmd.c`
Herramientas	`Tools/mkfw.py`, `Tools/mkfw_casos.py`, `Tools/verificar_commit_en_ram.sh`
Documentación viva	`CLAUDE.md`, sección *“Actualizar el firmware desde la microSD”*